МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний аерокосмічний університет
«Харківський авіаційний інститут»

Факультет систем управління літальних апаратів
Кафедра систем управління літальних апаратів

Лабораторна робота № 9

з дисципліни «Алгоритмізація та програмування»
на тему «Робота з рядками на С ++»

ХАІ.301. електрична інженерія. 319а. номер в списку 10 ЛР

Виконав студент гр. _____319а_____

_____Аріна_КУШАКОВА_____
(підпис, дата) (П.І.Б.)

Перевірив
_____ к.т.н., доц. Олена ГАВРИЛЕНКО
(підпис, дата) (П.І.Б.)

2026

МЕТА РОБОТИ

Вивчити теоретичний матеріал з основ роботи з низькорівневими рядками на C++ і документацію до класу string, а також алгоритми пошуку в рядку, а також реалізувати обробку рядків на C++ в середовищі QtCreator.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1.

A. Вивчити по документації метод стандартного класу string відповідно до варіанту (див. табл.1).

B. Визначити функцію, що виконує ті ж дії, що і вивчений метод класу string. Вихідний рядок передати першим параметром (масив символів). Для реалізації методу не використовувати функції обробки рядків зі стандартних бібліотек. C. Викликати свій метод і метод string аналогічно прикладам коду, наведеними в дод.А. *Перед викликом ввести з консолі один рядок і зберегти в масиві символів і змінній типу string.

Завдання 2.

A.Описати функцію, що обробляє рядок відповідно до завдання з табл.2. Для реалізації можна використовувати функції обробки рядків зі стандартних бібліотек

B.Описати функцію, яка перевіряє, чи задовольняє рядок умовам завдання.

C.* Створити вихідний текстовий файл, що містить не менше 10 різних рядків.

D.Використовуючи функції 2.A і 2.B, обробити рядок / * текстовий файл рядок за рядком. Додаткові дані ввести з консолі.

E. Отриманий результат записати у вихідний файл. Завдання 3.

Завдання 1-2 реалізувати окремими функціями без параметрів, у функції main() організувати меню для багаторазового виконання завдань.

Структурувати проєкт програми: винести заголовки і реалізацію функцій в окремі .h та .cpp файли.

Завдання 4. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок за

допомогою наступних промптів:

1) «Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»

2) «Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку»

3) «Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань»

ВИКОНАННЯ РОБОТИ

Завдання 1 — варіант 21.

Функція string& replace (size_t pos, size_t len, size_t n, char c);

Дана функція дозволяє замінити **len** символів рядку починаючи з позиції **pos** на символ **c**, продубльований **n** разів. Так як дана функція є методом класу **string**, для власної реалізації ми маємо додатково передавати посилання на рядок, що змінюється. Таким чином, власна функція матиме вигляд:

void replace(char **str, std::size_t pos, std::size_t len, std::size_t n, char c)

Інші параметри відповідають параметрам функції стандартної бібліотеки. В результаті виконання функція підміняє переданий рядок **str** на новий оброблений рядок. Алгоритм виконання функції представлено у графічному мигляді на рис. 1.

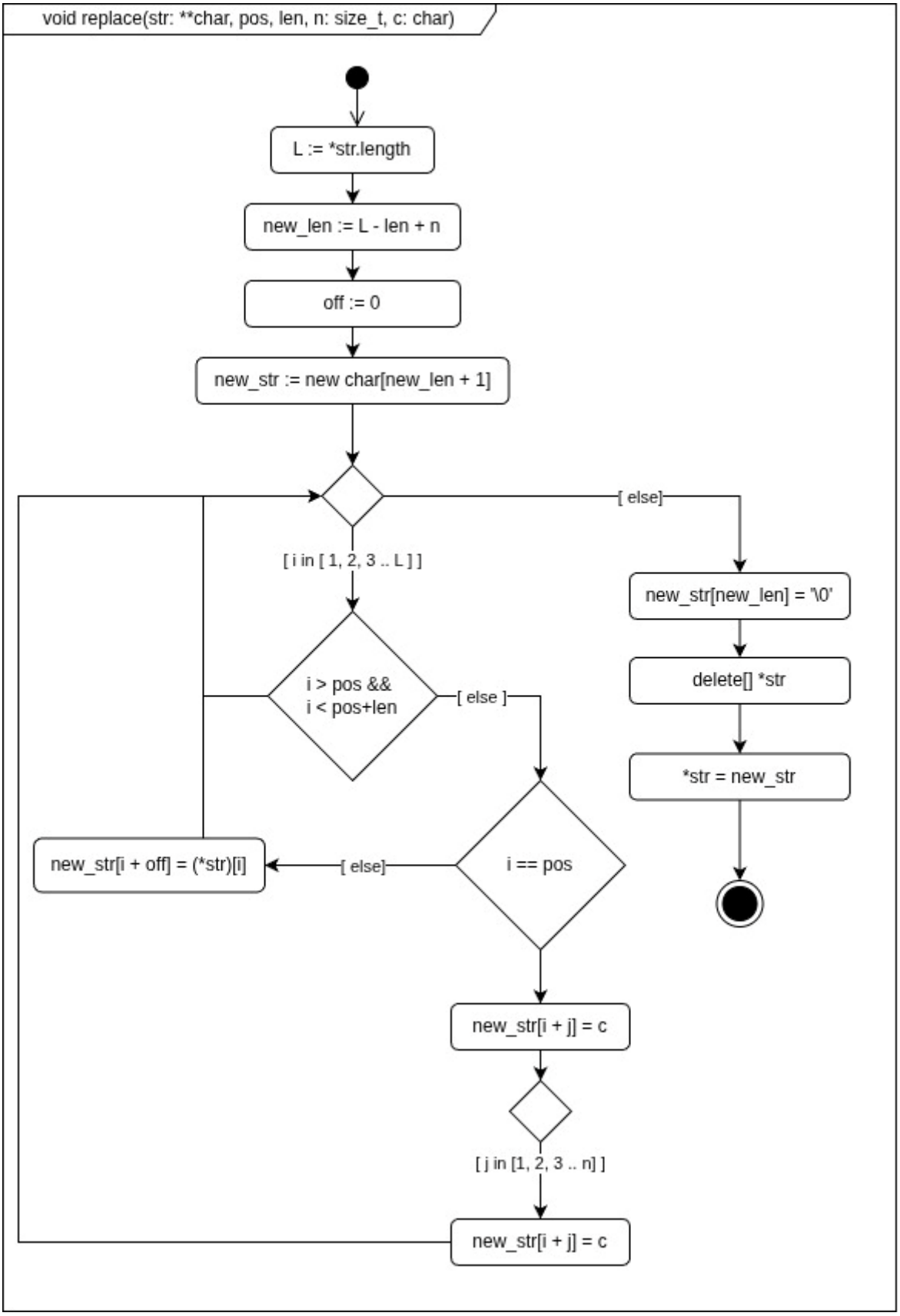

Рисунок 1 – Діаграма активності функції replace

Завдання 2 — варіант 32.

Дано рядки S і S_0. Знайти кількість входжень рядка S_0 в рядок S.

Функція розрахунку кількості входжень побудована на функції **find** стандартної бібліотеки, що дозволяє знайти першу позицію входження підрядка в рядок, починаючи з певної позиції. Маючи вхідний рядок, ми циклічно шукаємо наступну позицію входження, збільшуємо лічильник, відступаємо від неї на один символ та продовжуємо пошук з цієї позиції. Алгоритм виконання функції представлено у графічному мигляді на рис. 2.

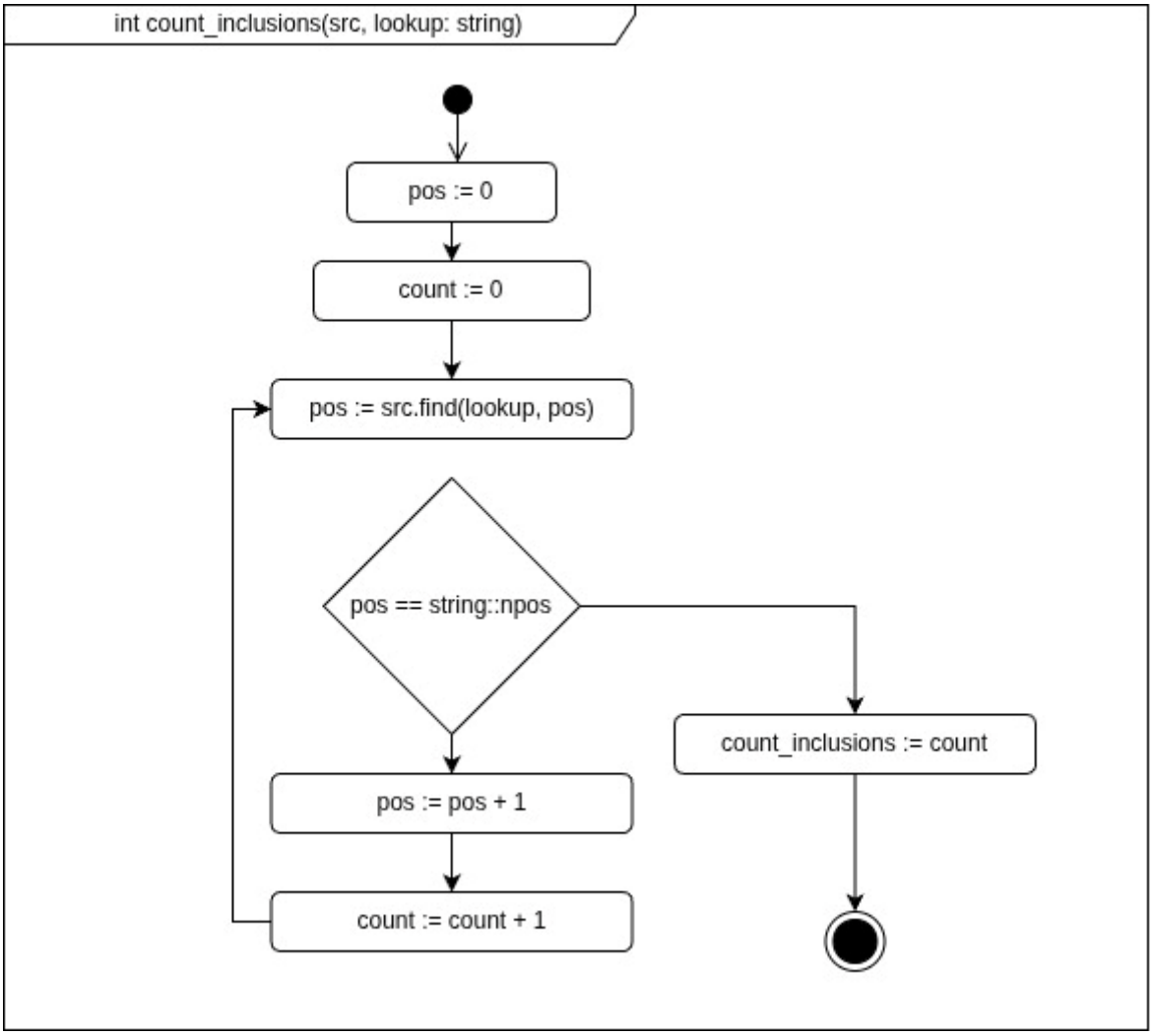

Рисунок 1 – Діаграма активності функції count_inclusions

Лістинг файлів для виконання завдання наведено в дод. Б (стор. 12).

Лістинг коду вирішення задач наведено в дод. А (стор. 7).

ВИСНОВКИ

Вивчили теоретичний матеріал з основ роботи з низькорівневими рядками на C++ і документацію до класу string, а також алгоритми пошуку в рядку, а

також реалізувати обробку рядків на C++ в середовищі QtCreator.

ДОДАТОК А
Лістинг коду програми

Файл **task1.h**

```
#include <cstddef>

// Завдання 21

// Зчитування довжини рядка з масиву символів
std::size_t get_len(const char *str);

// Заміна підрядка
void replace(char **str, std::size_t pos, std::size_t len, std::size_t n,
             char c);

// Повне завдання 1
void task1();
```

Файл **task1.cpp**

```
#include "task1.h"
#include <cstring>
#include <iostream>
#include <string>

// Зчитування довжини рядка з масиву символів
// Вважається, що рядок збережено в коректному форматі
// (закінчення вказано як \0)
std::size_t get_len(const char *str) {
  std::size_t len = 0;
  while (str[len] != '\0')
    len++;
  return len;
}

// Заміна підрядка
// str - посилання на вхідний рядок
// pos - позиція, з якої починати заміну
// len - кількість символів, що слід замінити
// n -  кількість символів, що слід вставити
// c - символ, що слід вставити
void replace(char **str, std::size_t pos, std::size_t len, std::size_t n,
             char c) {
  std::size_t L = get_len(*str);     // Отримуємо довжину рядку
  std::size_t new_len = L - len + n; // Вираховуємо нову довжину рядку
```

```
  int off = 0;
  char *new_str =
      new char[new_len + 1]; // Створюємо новий масив символів для заміни
  for (int i = 0; i < L; i++) {
    if (i > pos &&
        i < pos +
                len) // Якщо символ лежить в проміжку для заміни, ігноруємо його
      continue;
    else if (i == pos) {          // Якщо ми дійшли до першого символа заміни
      off = n - len;              // Змінюємо зміщення копіювання
      for (int j = 0; j < n; j++) // Додаємо n символів c до масиву
        new_str[i + j] = c;
    } else // Інакше додаємо символ старого масиву до нового
      new_str[i + off] = (*str)[i];
  }
  new_str[new_len] = '\0'; // Додаємо символ закінчення рядку
  delete[] *str;           // Очищуємо пам'ять
  *str = new_str;          // Підімняємо старий рядок на новий
}

// Повне завдання 1
void task1() {
  // Визначення змінних
  int max_len = 50;
  std::size_t pos = 0;
  std::size_t len = 0;
  std::size_t n = 0;
  char c = ' ';
  char *str = new char[max_len];
  // Зчитуємо дані з консолі
  std::cout << "Введіть рядок (до 50 символів): ";
  std::cin.getline(str, max_len);
  std::string base = str;
  std::cout << "Введіть початкову позицію: ";
  std::cin >> pos;
  std::cout << "Введіть кількість символів для заміни: ";
  std::cin >> len;
  std::cout << "Введіть кількість символів для вставки: ";
  std::cin >> n;
  std::cout << "Введіть символ для вставки: ";
  std::cin >> c;
  // Власна функція заміни
  replace(&str, pos - 1, len, n, c);
  std::cout << std::endl << "Результат заміни: " << str << std::endl;
  // Системна функція заміни
  std::cout << "Результат заміни (стандартна бібліотека): "
            << base.replace(pos - 1, len, n, c) << std::endl;
}
```

Файл **task2.h**

```
#include <cstddef>
#include <string>

// Розрахунок кількості підрядків
int count_inclusions(std::string src, std::string lookup);

// Перевірка правильності вводу
bool is_valid(std::string str);

// Повне завдання 2
void task2();
```

Файл **task2.cpp**

```
#include "task2.h"
#include <fstream>
#include <iostream>

// Обчислення кількості входжень підрядка lookup в рядок str
int count_inclusions(std::string src, std::string lookup) {
  std::size_t pos = 0; // Остання позиція входження
  int count = 0;       // Кількість входжень
  while (true) {
    pos = src.find(lookup, pos); // Шукаємо позицію входження
    if (pos ==
        std::string::npos) // Якщо позицію не визначено, завершуємо роботу
      break;
    pos += 1; // При вдалому визначенні, переходимо на 1 символ для уникнення
              // знаходження того ж підрядку
    count++;  // Збільшуємо кількість входжень
  }
  return count;
}

// Перевірка правильності вводу
bool is_valid(std::string str) {
  if (!str[0])
    return false; // Якщо рядок порожній, повертаємо false
  std::string allowed = "0123456789_ "
                        "AaBbCcDdEeFfGgHhIiJjKkLlMmNnOoPpQqRrSsTtUuVvWwXxYyZz"
                        "АБВГҐДЕЄЖЗИІЇЙКЛМНОПРСТУФХЦЧШЩЬЮЯ"
                        "абвгґдеєжзиіїйклмнопрстуфхцчшщьюя"; // Доступні символи
  for (int i = 0; str[i]; i++) {
    if (allowed.find(str[i]) ==
        std::string::npos) // Якщо символ не в списку доступних, повертаємо
```

```
                            // false
      return false;
  }
  return true;
}

// Повне завдання 2
void task2() {
  std::ifstream file("data.txt");
  if (!file) {
    std::cerr << "Не вдалося відкрити файл data.txt" << std::endl;
    return;
  }
  std::string src;
  std::string lookup;
  std::ofstream output("out.txt");
  if (!output) {
    std::cerr << "Не вдалося відкрити файл out.txt" << std::endl;
    return;
  }
  while (true) {
    if (!std::getline(file, src))
      break;
    if (!std::getline(file, lookup))
      break;
    if (!is_valid(src)) {
      std::cout
          << "Невірний формат рядку: " << src << std::endl
          << "Текст має містити лише англійські літери, кирилицю, числа або '_'"
          << std::endl;
      break;
    }
    if (!is_valid(lookup)) {
      std::cout
          << "Невірний формат рядку: " << lookup << std::endl
          << "Текст має містити лише англійські літери, кирилицю, числа або '_'"
          << std::endl;
      break;
    }
    output << "Рядок: " << src << std::endl;
    output << "Підрядок: " << lookup << std::endl;
    output << "Входжень: " << count_inclusions(src, lookup) << std::endl;
  }
  file.close();
  output.close();
}
```

Файл **main.cpp**

```
#include "task1.h"
#include "task2.h"
#include <iostream>

int main() {
  int n;
  // Головне меню
  while (n != 0) {
    std::cout << std::endl << std::endl;
    std::cout << "[      Головне Меню      ]" << std::endl << std::endl;
    std::cout << "[ 1 ] Завдання 1" << std::endl;
    std::cout << "[ 2 ] Завдання 2" << std::endl;
    std::cout << "[ 0 ] Вихід" << std::endl << std::endl;
    std::cout << "Оберіть пункт меню: ";
    std::cin >> n;
    std::cin.ignore();
    switch (n) {
    case 1:
      std::cout << std::endl << std::endl;
      std::cout << "[      Завдання 1        ]" << std::endl << std::endl;
      task1();
      break;
    case 2:
      std::cout << std::endl << std::endl;
      std::cout << "[      Завдання 2        ]" << std::endl << std::endl;
      task2();
      break;
    case 0:
      return 0;
    default:
      std::cout << std::endl
                << std::endl
                << "Невірно обрано пункт меню. Спробуйте знову";
      break;
    }
    // Очікування на Enter
    std::cout << std::endl
              << std::endl
              << "Натисніть Enter для повернення до головного меню";
    std::cin.get();
    std::cin.ignore();
  }
}
```

ДОДАТОК Б
Вхідні та вихідні файли завдань

Б.1 — Вхідний файл завдання **2** — data.txt

```
hello world hello
hello
aaaaa
aa
перевірка перевірки
перевірк
abcdefg
x
exact
exact
```

Б.2 — Вихідний файл завдання **2** — out.txt

```
Рядок: hello world hello
Підрядок: hello
Входжень: 2
Рядок: aaaaa
Підрядок: aa
Входжень: 4
Рядок: перевірка перевірки
Підрядок: перевірк
Входжень: 2
Рядок: abcdefg
Підрядок: x
Входжень: 0
Рядок: exact
Підрядок: exact
Входжень: 1
```

ДОДАТОК В

Діалог з ШІ для самоаналізу

Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції Файл task1.h #include <cstddef> // Завдання 21 // Зчитування довжини рядка з масиву символів std::size_t get_len(const char *str); // Заміна підрядка void replace(char **str, std::size_t pos, std::size_t len, std::size_t n, char c); // Повне завдання 1 void task1(); Файл task1.cpp #include "task1.h" #include <cstring> #include <iostream> #include <string> // Зчитування довжини рядка з масиву символів // Вважається, що рядок збережено в коректному форматі // (закінчення вказано як \0) std::size_t get_len(const char *str) { std::size_t len = 0; while (str[len] != '\0') len++; return len; } // Заміна підрядка // str - посилання на вхідний рядок // pos - позиція, з якої починати заміну // len - кількість символів, що слід замінити // n - кількість символів, що слід вставити // c - символ, що слід вставити void replace(char **str, std::size_t pos, std::size_t len, std::size_t n, char c) { std::size_t L = get_len(*str); // Отримуємо довжину рядку std::size_t new_len = L - len + n; // Вираховуємо нову довжину рядку int off = 0; char *new_str = new char[new_len + 1]; // Створюємо новий масив символів для заміни for (int i = 0; i < L; i++) { if (i > pos && i < pos + len) // Якщо символ лежить в проміжку для заміни, ігноруємо його continue; else if (i == pos) { // Якщо ми дійшли до першого символа заміни off = n - len; // Змінюємо зміщення копіювання for (int j = 0; j < n; j++) // Додаємо n символів c до масиву new_str[i + j] = c; } else // Інакше додаємо символ старого масиву до нового new_str[i + off] = (*str)[i]; } new_str[new_len] = '\0'; // Додаємо символ закінчення рядку delete[] *str; // Очищуємо пам'ять *str = new_str; // Підмняємо старий рядок на новий } // Повне завдання 1 void task1() { // Визначення змінних int max_len = 50; std::size_t pos = 0; std::size_t len = 0; std::size_t n = 0; char c = ' '; char *str = new char[max_len]; // Зчитуємо дані з консолі std::cout << "Введіть рядок (до 50 символів): "; std::cin.getline(str, max_len); std::string base = str; std::cout << "Введіть початкову позицію: "; std::cin >> pos; std::cout << "Введіть кількість символів для заміни: "; std::cin >> len; std::cout << "Введіть кількість символів для вставки: "; std::cin >> n; std::cout << "Введіть символ для вставки: "; std::cin >> c; // Власна функція заміни replace(&str, pos - 1, len, n, c); std::cout << std::endl << "Результат заміни: " << str << std::endl; // Системна функція заміни std::cout << "Результат заміни (стандартна бібліотека): " << base.replace(pos - 1, len, n, c) << std::endl; } Файл task2.h #include <cstddef> #include <string> // Розрахунок кількості підрядків int count_inclusions(std::string src, std::string lookup); // Перевірка правильності вводу bool is_valid(std::string str); // Повне завдання 2 void task2(); Файл task2.cpp #include "task2.h" #include <fstream> #include <iostream> // Обчислення кількості входжень підрядка lookup в рядок

str int count_inclusions(std::string src, std::string lookup) { std::size_t pos = 0; // Остання позиція входження int count = 0; // Кількість входжень while (true) { pos = src.find(lookup, pos); // Шукаємо позицію входження if (pos == std::string::npos) // Якщо позицію не визначено, завершуємо роботу break; pos += 1; // При вдалому визначенні, переходимо на 1 символ для уникнення // знаходження того ж підрядку count++; // Збільшуємо кількість входжень } return count; } // Перевірка правильності вводу bool is_valid(std::string str) { if (!str[0]) return false; // Якщо рядок порожній, повертаємо false std::string allowed = "0123456789_ " "AaBbCcDdEeFfGgHhIiJjKkLlMmNnOoPpQqRrSsTtUuVvWwXxYyZz" "АБВГҐДЕЄЖЗИІЇЙКЛМНОПРСТУФХЦЧШЩЬЮЯ" "абвгґдеєжзиіїйклмнопрстуфхцчшщьюя"; // Доступні символи for (int i = 0; str[i]; i++) { if (allowed.find(str[i]) == std::string::npos) // Якщо символ не в списку доступних, повертаємо // false return false; } return true; } // Повне завдання 2 void task2() { std::ifstream file("data.txt"); if (!file) { std::cerr << "Не вдалося відкрити файл data.txt" << std::endl; return; } std::string src; std::string lookup; std::ofstream output("out.txt"); if (!output) { std::cerr << "Не вдалося відкрити файл out.txt" << std::endl; return; } while (true) { if (!std::getline(file, src)) break; if (!std::getline(file, lookup)) break; if (!is_valid(src)) { std::cout << "Невірний формат рядку: " << src << std::endl << "Текст має містити лише англійські літери, кирилицю, числа або '_'" << std::endl; break; } if (!is_valid(lookup)) { std::cout << "Невірний формат рядку: " << lookup << std::endl << "Текст має містити лише англійські літери, кирилицю, числа або '_'" << std::endl; break; } output << "Рядок: " << src << std::endl; output << "Підрядок: " << lookup << std::endl; output << "Входжень: " << count_inclusions(src, lookup) << std::endl; } file.close(); output.close(); } Файл main.cpp #include "task1.h" #include "task2.h" #include <iostream> int main() { int n; // Головне меню while (n != 0) { std::cout << std::endl << std::endl; std::cout << "[Головне Меню]" << std::endl << std::endl; std::cout << "[1] Завдання 1" << std::endl; std::cout << "[2] Завдання 2" << std::endl; std::cout << "[0] Вихід" << std::endl << std::endl; std::cout << "Оберіть пункт меню: "; std::cin >> n; std::cin.ignore(); switch (n) { case 1: std::cout << std::endl << std::endl; std::cout << "[Завдання 1]" << std::endl << std::endl; task1(); break; case 2: std::cout << std::endl << std::endl; std::cout << "[Завдання 2]" << std::endl << std::endl; task2(); break; case 0: return 0; default: std::cout << std::endl << std::endl << "Невірно обрано пункт меню. Спробуйте знову"; break; } // Очікування на Enter std::cout << std::endl << std::endl << "Натисніть Enter для повернення до головного меню"; std::cin.get(); std::cin.ignore(); } }

Добре, давай перевіримо твоє розуміння — як коду, так і теорії. Питання середнього рівня, з акцентом на мислення, а не просто запам’ятовування.

⬙ Тестові питання (1 правильна відповідь)

1. Яка асимптотична складність функції `get_len`?

A) O(1)
B) O(log n)
C) O(n)
D) O(n^2)

---

2. Яка потенційна проблема є у функції `replace`?

A) Вона не виділяє пам’ять
B) Можливий вихід за межі масиву при некоректних `pos` і `len`
C) Вона не змінює рядок
D) Вона працює тільки зі `std::string`

---

3. Що означає `std::string::npos` у функції `count_inclusions`?

A) Позиція першого символу
B) Помилка компіляції
C) Ознака того, що підрядок не знайдено
D) Кінець файлу

---

4. Чому у `count_inclusions` використовується `pos += 1`, а не `pos += lookup.length()`?

A) Це помилка
B) Щоб рахувати перекриваючі входження
C) Для оптимізації швидкості
D) Це не має значення

---

5. Що не так із перевіркою в `is_valid`?

```
if (!str[0])
```

A) Це синтаксична помилка
B) Це не перевіряє порожній рядок коректно для `std::string`
C) Це перевіряє тільки цифри
D) Це перевіряє лише останній символ

⬙ Відкриті питання

1.

У функції `replace` використовується `char** str`.
Поясни, чому тут використано подвійний вказівник, і які альтернативи існують у сучасному C++.

2.

Проаналізуй функцію `replace` і скажи:

- що станеться, якщо `pos > довжини рядка`?
- як би ти зробив її безпечнішою?

3.

Порівняй підходи:

- робота з `char*`
- використання `std::string`

Коли який підхід доцільніший з точки зору:

- безпеки
- продуктивності
- зручності

4.

Функція `count_inclusions` рахує входження підрядка.
Як зміниться результат, якщо замінити:

```
pos += 1;
```

на

```
pos += lookup.length();
```

Наведи приклад.

5.

У `main()` змінна `n` не ініціалізована:

```
int n;
while (n != 0)
```

- Чому це проблема?

- Які можуть бути наслідки під час виконання?
- Як це правильно виправити?

Якщо хочеш — можу після твоїх відповідей провести “реальний захист”: вказати слабкі місця і поставити додаткові уточнюючі питання 👀

Відповіді на тестові питання: 1) B - O(n) 2) B - Можливий вихід за межі масиву при некоректних pos і len 3) C - Ознака того, що підрядок не знайдено 4) B - Щоб рахувати перекриваючі входження 5) B - це не перевіряє порожній рядок коректно для std::string Відповіді на відкриті питання: 1) Функція replace підміняє вхідний рядок, тому ми використовуємо вказівник на масив символів. Це дозволяє нам "переписати" рядок, підмінивши вказівник на новий рядок. Зазвичай, для масивів символів ми використовуємо char*, значення якого є вказівником на перший елемент масиву. Використовуючи char**, ми можемо змінити цей вказівник на вказівник до іншого масиву символів. 2) При значенні pos більше за довжину рядка, програма пропрацює символи по довжині L, проте, через те, що умова (i == pos) ніколи не спрацює, не буде опрацьовано зміщення для коректної обробки символів після області заміни. Таким чином, при значеннях n > len, ми матимемо частину наприкінці рядка, що братиме значення з-за меж початкового масиву символів. Щоб виправити цю проблему, слід перевіряти, що pos та pos + len лежать в межах вхідного масиву. 3) Порівняно з char*, std::string є набагато зручнішим. З точки зору безпеки, цей тип даних автоматично опрацьовує такі речі як вихід за межі масиву, що є великою проблемою при роботі з char*. Також, він має велику кількість вбудованих методів, таких як find чи replace, що були використані в лабораторній роботі, що як збільшує продуктивність, так і полегшує роботу з рядками. 4) При заміні pos += 1 на pos += lookup.length() ми перестанемо враховувати перекриваючі підрядки. Наприклад, маючи рядок "aaaaa", та шукаючи підрядок "aa", ми б мали такий алгоритм дій: Перше входження - позиція 0; (AAaaa); зміщуємося на 2 символи Друге входження - позиція 2; (aaAAa); зміщуємося на 2 символи Третє входження не знайдено; завершуємо роботу При pos +=1 маємо: Перше входження - позиція 0; (AAaaa); зміщуємося на 1 символ Друге входження - позиція 0; (aAAaa); зміщуємося на 1 символ Третє входження - позиція 0; (aaAAa); зміщуємося на 1 символ Четверте входження - позиція 0; (aaaAA); зміщуємося на 1 символ П'яте входження не знайдено; завершуємо роботу Можна побачити, що результат сильно відрізняється залежно від методу. Можна

сказати, що саме операція додавання визначає тип пошуку підрядку - перекриваючий чи ні. 5) Якщо не ініціалізувати змінну n, ми лише виділяємо під неї місце в пам'яті, не очищуючи його. Через це змінна отримує "випадкове" значення, підбираючи старі дані з пам'яті. В нашому випадку, може статися таке, що змінна отримає значення 0, через що цикл одразу завершить роботу. Цього можна уникнути двома шляхами: надати значення n, наприклад, 1, що дозволить увійти у цикл, де ця змінна одразу зчитається з консолі, або змінити тип циклу на do ... while, що виконає тіло циклу щонайменш один раз, дозволивши перезаписати старе "брудне" значення змінної

Добре, відповіді сильні — видно, що ти реально розумієш код. Але я буду трохи “прискіпливим викладачем” і розберу детально 👇

⬙ Тестові питання

1) ✕

Ти обрав **B — O(n)**, але написано **B = O(log n)**

☞ Правильна відповідь: **C — O(n)**

✓ По суті ти мислиш правильно, просто помилка у виборі варіанту.

---

2) ✅

Абсолютно правильно.
Тут ключове — **відсутність перевірки меж**, що може призвести до UB (undefined behavior).

---

3) ✅

Чітко і правильно.

---

4) ✅

Дуже добре — ти зрозумів ідею **перекриваючих входжень**, це важливий момент.

---

5) ✅ (з нюансом)

Твоя відповідь правильна, але можна глибше:

```
if (!str[0])
```

☞ Це працює, але:

- виглядає як C-style
- неочевидно для `std::string`

✓ Краще:

```
if (str.empty())
```

⬙ Відкриті питання

1) ✅

Дуже хороше пояснення.

✓ Ти правильно сказав:

- чому потрібен `char**`
- що ми змінюємо сам вказівник

➕ Що можна було додати:

- сучасна альтернатива:
  - `std::string&`
  - або навіть `std::string` (повернення значення)

---

2) ⚠ (частково правильно, але є неточність)

Ти правильно помітив проблему з `pos`.

Але ось це місце:

"ми матимемо частину наприкінці рядка, що братиме значення з-за меж початкового масиву"

! Не зовсім так.

☞ Насправді:

- цикл іде до `L`
- доступу за межі **старого масиву не буде**
- але:
  - **новий рядок буде частково неініціалізований**
  - або логіка заміни буде просто зламана

✓ Ключова проблема:

- **порушення логіки, а не обов'язково вихід за межі**

---

3) ⚠ (є помилка в теорії)

Тут важливий момент:

"std::string ... збільшує продуктивність"

! Це НЕ зовсім правильно.

☞ Насправді:

- `std::string`:
  - ✓ безпечніший
  - ✓ зручніший
  - ✗ не завжди швидший

✓ Правильніше:

- `char*` → потенційно швидший (менше абстракцій)
- `std::string` → трохи повільніший, але:
  - значно безпечніший
  - менш схильний до помилок

---

4) ✅

Ідеально.

✓ Ти:

- дав правильний приклад
- показав різницю
- пояснив концепцію

👉 Це рівень “добре розуміє алгоритм”.

---

5) ✅

Дуже хороша відповідь.

✓ Ти покрив:

- причину (сміття в пам’яті)
- наслідки
- два варіанти виправлення

➕ Маленьке доповнення:
краще просто:

```
int n = -1;
```

⬙ Загальна оцінка (як на захисті)

☞ **Рівень: вище середнього (≈ 85–90%)**

Сильні сторони:

- добре розумієш роботу з пам’яттю
- правильно мислиш алгоритмічно
- вмієш пояснювати логіку

Слабкі місця:

- іноді неточності у формулюваннях
- трохи плутаєшся в теоретичних деталях (продуктивність, UB) ⬙ Додаткове питання (як на реальному захисті 😊)

У функції `replace` є фрагмент:

```
for (int i = 0; i < L; i++)
```

☞ Питання:

- Чому використання `int`, а не `std::size_t`, може бути проблемою?
- У яких ситуаціях це реально “вистрелить”?

Відповідай — і я скажу, чи це вже рівень “відмінно” 👀